Integrating a Zoho Creator application with another Zoho service using connections and APIs

Insurance Claim Request & Processing — Zoho Calendar Integration Use Case

Name: Raghavi S **Roll No:** 241401074

1. Aim

To integrate a Zoho Creator insurance claim request and processing application with Zoho Calendar using microservice connections, conditional workflows, and Deluge scripting, so that every valid claim submission automatically schedules a claim inspection (surveyor visit) in the calendar.

2. Use Case Overview

In an insurance company, a policyholder raises a claim through an online Claim Request Form. The claims team must then verify the documents, schedule an inspection or assessment, and track the claim until settlement. Scheduling inspections manually causes delays and missed appointments. This experiment automates that step by linking the Zoho Creator application to Zoho Calendar.

Item	Details
Application	Insurance Claim Management (Zoho Creator)
Form	Claim_Request_Form
Target service	Zoho Calendar
Connection link name	zoho_calendar_connection
Workflow	Sync_To_Zoho_Calendar

3. Procedure

Phase A: Navigating to the Connection Panel

1. Log in to your Zoho Creator developer account and open the active Insurance Claim Request & Processing application.

2. In the left navigation panel under Develop, click Microservices to access the centralized integration environment.

3. Choose Connections and use Pick & Create to configure the Zoho Calendar service connection.

241401074

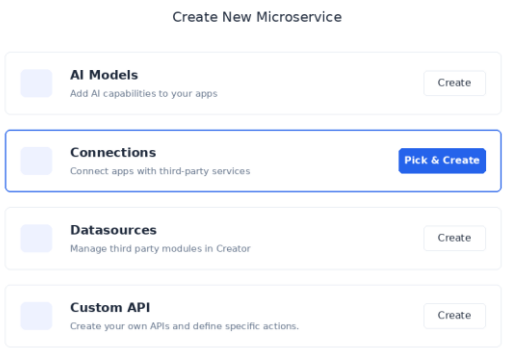


Figure 1. The Create New Microservice panel with Connections selected using Pick & Create.

Phase A: Creating the Zoho Calendar Connection

4. Select Zoho Calendar and create the connection using the link name **zoho_calendar_connection**.

5. Set Authorization Access to Administrators only, select Quick connect (Zoho credential), enable All Actions if available, and click Create and Authorize.

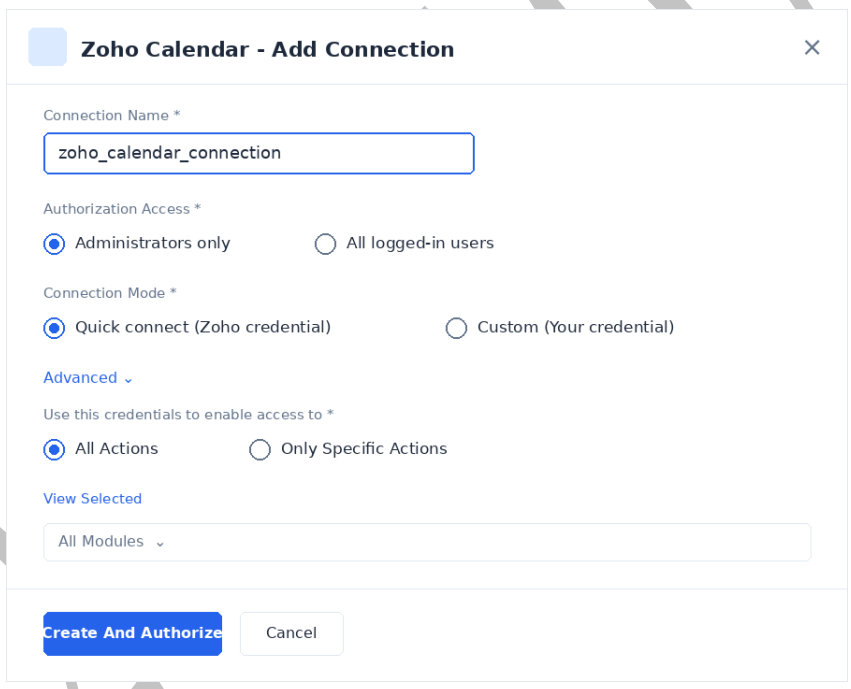


Figure 2. Zoho Calendar — Add Connection dialog with the connection name and authorization options.

Phase B: Verifying Scopes and Application References

6. Open the Zoho Calendar Connection asset to inspect its security and permission settings.

7. Confirm that the required permissions include Create Calendar, Create Event, and Get all Calendars.

8. In the Connection Access matrix, switch on access for Insurance Claim Management if required.

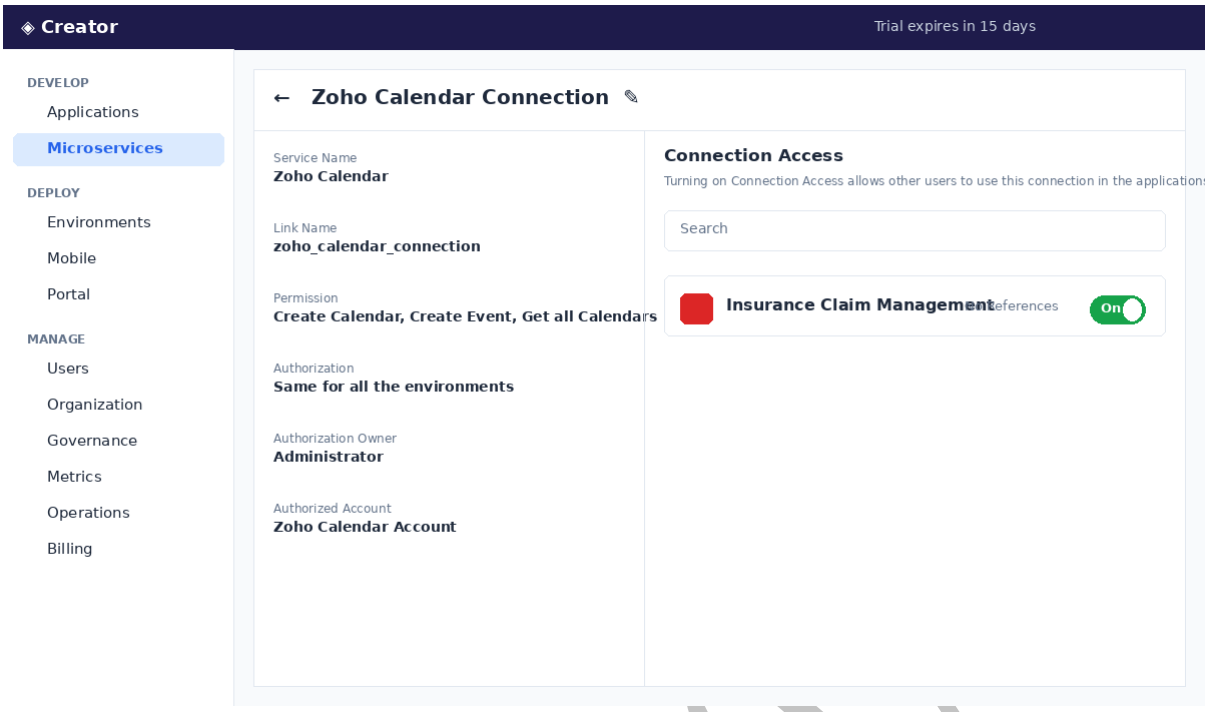


Figure 3. Connection management page showing the connection and application access settings for the insurance claim application.

Phase C: Setting Up Conditional Rules

9. Return to the Insurance Claim Management application, open the Claim_Request_Form workflow area, and create a form workflow named Sync_To_Zoho_Calendar.

10. Configure the trigger as **Created → Successful form submission**.

11. In the action builder, change the run setting from Always to **on condition**. Select **Claim_Declaration**, choose the equals comparison, and set the value to the checkbox choice “I declare the claim details are true”.

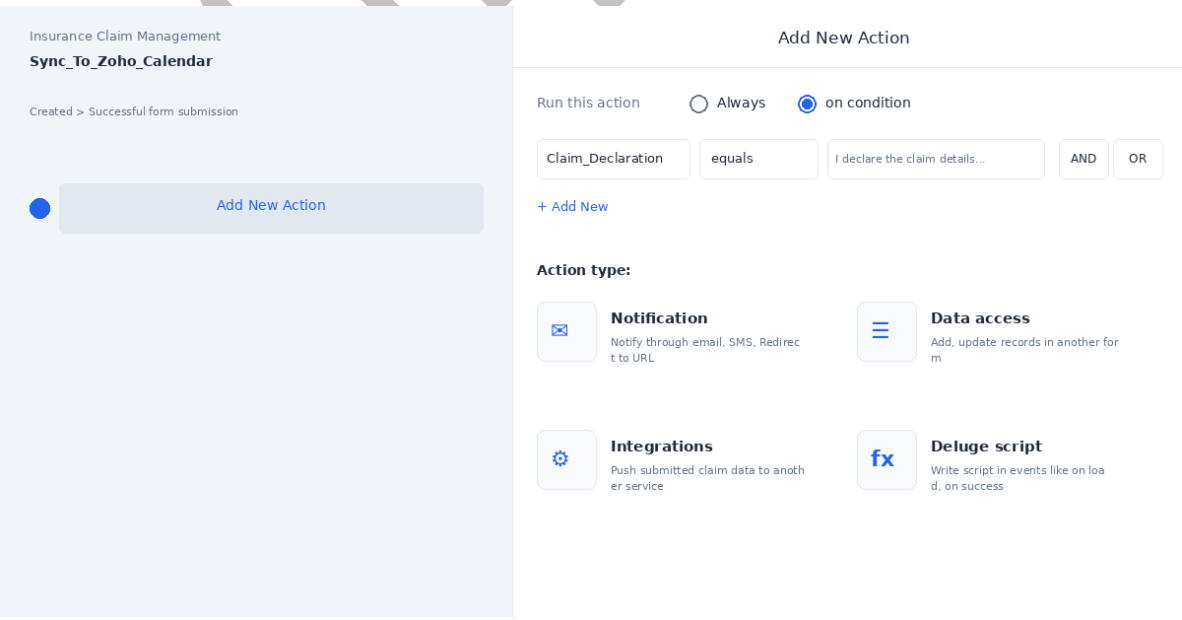


Figure 4. Add New Action panel with the condition builder and available action types.

The condition ensures that the calendar event is created only for claims where the policyholder has accepted the declaration. Incomplete or unverified claim requests are not scheduled for inspection.

Setting	Value	Purpose
Trigger	Created → Successful form submission	Runs when a new claim is saved
Run this action	on condition	Filters claims
Condition field	Claim_Declaration equals “I declare the claim details are true”	Accepts only declared claims
Action type	Deluge script (fx)	Calls the Calendar API

Phase D: Compiling the Deluge Script Code

12. Select **Deluge script (fx)** from the action type grid to open the embedded Deluge editor.

13. Map the policyholder email, claim type, inspection date, and inspection slot into a claim event data map. Use **invokeurl** with the authorized connection link name **zoho_calendar_connection** to call the Zoho Calendar API.

14. Click Save to validate and save the script. Verify the API response and test with a sample claim request.



Figure 5. Deluge Script editor for the claim inspection calendar integration workflow.

Script used in the workflow:

```
claimEvent = Map();
claimEvent.put("title","Claim Inspection - " +
input.Claim_Number);
claimEvent.put("description","Type: " + input.Claim_Type + " |
Policy: " + input.Policy_Number);
```

```
startDT = input.Inspection_Date.toString("yyyyMMdd") + "T" +
input.Inspection_Slot;
claimEvent.put("dateandtime",{"timezone":"Asia/Kolkata","start":st
artDT});
claimEvent.put("attendees",{{"email":input.Policyholder_Email}});
response = invokeurl
[
    url :
"https://calendar.zoho.com/api/v1/calendars/<calendar_uid>/events"
    type : POST
    parameters : {"eventdata":claimEvent.toString()}
    connection : "zoho_calendar_connection"
];
info response;
```

Phase E: Testing the Workflow

15. Open the Claim_Request_Form in access mode and submit a sample claim with the declaration checkbox selected.

16. Open Zoho Calendar and confirm that a Claim Inspection event appears at the chosen date and slot, and that the API response shows success.

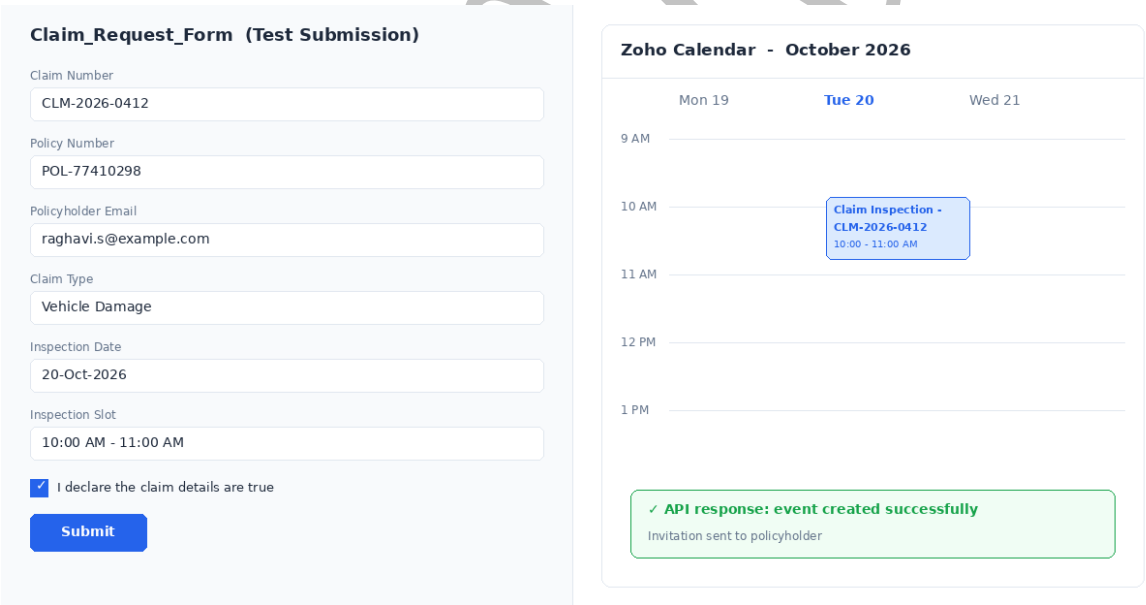


Figure 6. Sample claim submission and the resulting inspection event in Zoho Calendar.

4. Output & System Verification

The integration connection is configured in Zoho Creator. The following screenshot shows the Microservices dashboard with the Zoho Calendar connection and its current connection status.

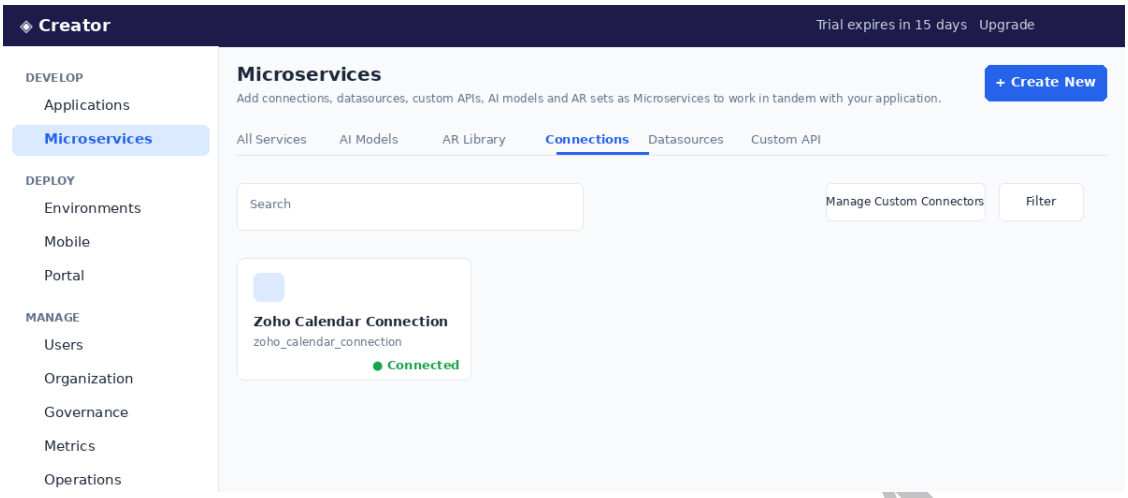


Figure 7. Current Microservices Connections dashboard showing zoho_calendar_connection with Connected status.

Verification checklist

- Connection link name: **zoho_calendar_connection**
- Connection status: **Connected**
- Workflow name: **Sync_To_Zoho_Calendar**
- Trigger: **Created → Successful form submission**
- Condition: **Claim_Declaration matches the declaration choice**
- Deluge action: **calls Zoho Calendar API through the configured connection**

5. Result

A claim request submitted with the declaration accepted triggers the Sync_To_Zoho_Calendar workflow. The Deluge script builds the event data from the claim details and creates an inspection event in Zoho Calendar through the authorized connection, notifying the policyholder by invitation.

6. Conclusion

This experiment demonstrates how to integrate a Zoho Creator application with another Zoho service using a microservice connection, a conditional workflow, and Deluge scripting. The workflow is designed to use insurance claim request details to create a corresponding Zoho Calendar inspection event after the required declaration condition is satisfied, reducing manual scheduling effort and speeding up claim processing.

241401074